

Análise de Complexidade de Algoritmos Recursivos e Divisão e Conquista: mergesort

Yuri Kaszubowski Lopes

UDESC

Notação assintótica

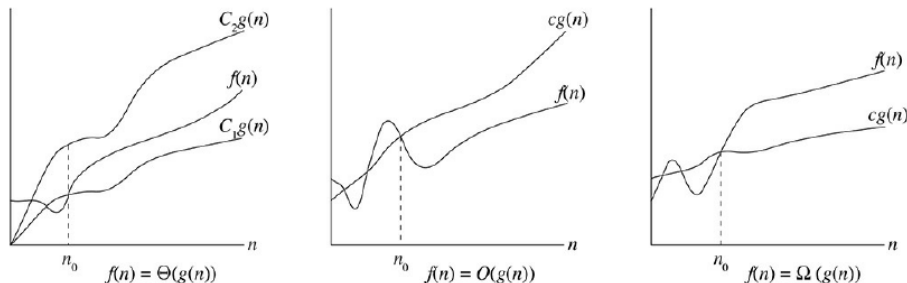


Figura: Exemplos de crescimento assintótico. Fonte: CORMEN, Thomas H. et. al. Introduction to Algorithms. 3 ed. The MIT Press, 2009

Notação assintótica

- Tempo de execução $T(n)$: número de passos primitivos
- $\Theta(g(n)) = \{f(n) : \exists c_1, c_2 \text{ e } n_0 \in \mathbb{R}^+ : 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0\}$
- $O(g(n)) = \{f(n) : \exists c \text{ e } n_0 \in \mathbb{R}^+ : 0 \leq f(n) \leq c g(n) \forall n \geq n_0\}$
- $\Omega(g(n)) = \{f(n) : \exists c \text{ e } n_0 \in \mathbb{R}^+ : 0 \leq c g(n) \leq f(n) \forall n \geq n_0\}$
- $o(g(n)) = \{f(n) : \forall c \in \mathbb{R}^+ \exists n_0 \in \mathbb{R}^+ : 0 \leq f(n) < c g(n) \forall n \geq n_0\}$
- $\omega(g(n)) = \{f(n) : \forall c \in \mathbb{R}^+ \exists n_0 \in \mathbb{R}^+ : 0 \leq c g(n) < f(n) \forall n \geq n_0\}$

Notação assintótica

Notações assintóticas comuns

- $\Theta(1)$
- $\Theta(\lg n)$
- $\Theta(n)$
- $\Theta(n \lg n)$
- $\Theta(n^2)$
- $\Theta(n^3)$
- $\Theta(n^c)$
- $2^{\Theta(n^c)}$

Termo dominante

- $T(n) = n^2 + n + n \lg n \rightarrow \Theta(n^2)$
- $T(n) = \lg n + 1 \rightarrow \Theta(\lg n)$

Ordenação por inserção

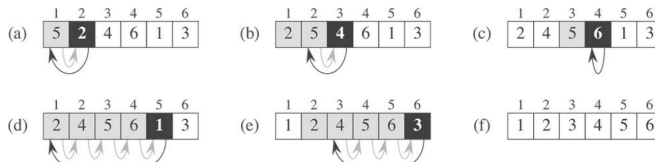


Figura: Exemplo do algoritmo de ordenação por inserção. Adaptado de: CORMEN, Thomas H. et. al. Introduction to Algorithms. 3 ed. The MIT Press, 2009

Ordenação por inserção: algoritmo, seu tempo de execução e análise

```
1 insertsort(A):  
2     para j = 2 até A.comprimento:  
3         chave = A[j]  
4         i = j - 1  
5         enquanto i > 0 e A[i] > chave  
6             A[i+1] = A[i]  
7             i = i - 1  
8         A[i+1] = chave  
9
```

Ordenação por inserção: algoritmo, seu tempo de execução e análise

```
1 insertsort(A):  
2     para j = 2 até A.comprimento:  
3         chave = A[j]  
4         i = j - 1  
5         enquanto i > 0 e A[i] > chave  
6             A[i+1] = A[i]  
7             i = i - 1  
8         A[i+1] = chave  
9
```

Qual o tempo de execução, $T(n)$?

Ordenação por inserção: algoritmo, seu tempo de execução e análise

	custo	vezes
1 insertsort(A):	c_1	n
2 para j = 2 até A.comprimento:	c_2	$n - 1$
3 chave = A[j]	c_3	$n - 1$
4 i = j - 1	c_4	$\sum_{j=2}^n t_j$
5 enquanto i > 0 e A[i] > chave	c_5	$\sum_{j=2}^n (t_j - 1)$
6 A[i+1] = A[i]	c_6	$\sum_{j=2}^n (t_j - 1)$
7 i = i - 1	c_7	$n - 1$
8 A[i+1] = chave		
9		

Qual o tempo de execução, $T(n)$?

Melhor caso: Se já ordenado $t_j = 1 \forall j = 2, 3, \dots, n$

$$\begin{aligned}T(n) &= c_1 n + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_7(n-1) \\&= c_1 n + (c_2 + c_3 + c_4 + c_7) \times (n-1) \\&= (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c_7)\end{aligned}$$

$an + b \Rightarrow$ função linear de $n \rightarrow \Omega(n)$

Ordenação por inserção: algoritmo, seu tempo de execução e análise

	custo	vezes
1 insertsort(A):	C_1	n
2 para j = 2 até A.comprimento:	C_2	$n - 1$
3 chave = A[j]	C_3	$n - 1$
4 i = j - 1	C_4	$\sum_{j=2}^n t_j$
5 enquanto i > 0 e A[i] > chave	C_5	$\sum_{j=2}^n (t_j - 1)$
6 A[i+1] = A[i]	C_6	$\sum_{j=2}^n (t_j - 1)$
7 i = i - 1	C_7	$n - 1$
8 A[i+1] = chave		
9		

Qual o tempo de execução, $T(n)$?

Pior caso: Inversamente ordenado $t_j = j \forall j = 2, 3, \dots, n$

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_3(n-1) + c_4\left(\frac{n(n+1)}{2} - 1\right) + c_5\left(\frac{n(n-1)}{2}\right) + c_6\left(\frac{n(n-1)}{2}\right) + c_7(n-1) \\ &= \left(\frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2}\right)n^2 + (c_1 + c_2 + c_3 + \frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2} + c_8)n - (c_2 + c_3 + c_4 + c_7) \end{aligned}$$

$an^2 + bn + c \Rightarrow$ função quadrática de $n \rightarrow \Theta(n^2)$, $O(n^2)$

Algoritmos Recursivos

Motivação

- Muitos algoritmos têm definição recursivas.
- O que acontece com uma entrada grande?
- Atende as necessidades?
- Solução correta?

Definições recursivas: Multiplicação de inteiros

$$a \times b$$

Definições recursivas: Multiplicação de inteiros

$$a \times b$$

$$a \times b = \underbrace{a + a + \dots + a}_{b \text{ times}}$$

Definições recursivas: Multiplicação de inteiros

$$a \times b$$

$$a \times b = \underbrace{a + a + \dots + a}_{b \text{ times}}$$

$$a \times b = \begin{cases} a + (a \times (b - 1)) & \text{se } b > 1 \leftarrow \text{operação básica: soma} \\ a & \text{se } b = 1 \leftarrow \text{caso base} \end{cases}$$

Definições recursivas: Multiplicação de inteiros

$$a \times b$$

$$a \times b = \underbrace{a + a + \dots + a}_{b \text{ times}}$$

$$a \times b = \begin{cases} a + (a \times (b - 1)) & \text{se } b > 1 \leftarrow \text{operação básica: soma} \\ a & \text{se } b = 1 \leftarrow \text{caso base} \end{cases}$$

```
1 mult(a, b):  
2     se b == 0:  
3         retorne 0;  
4     retorne a + mult(a, b - 1);
```

Número fatorial

$a!$

Número fatorial

$$a!$$

$$a! = \underbrace{a \times (a-1) \times (a-2) \dots \times 2 \times 1}_{a \text{ times}}$$

Número fatorial

$$a!$$

$$a! = \underbrace{a \times (a-1) \times (a-2) \dots \times 2 \times 1}_{a \text{ times}}$$

$$a! = \begin{cases} a \times (a-1)! & \text{se } a > 1 \\ 1 & \text{se } a = 1 \end{cases}$$

Número fatorial

$$a!$$

$$a! = \underbrace{a \times (a-1) \times (a-2) \dots \times 2 \times 1}_{a \text{ times}}$$

$$a! = \begin{cases} a \times (a-1)! & \text{se } a > 1 \\ 1 & \text{se } a = 1 \end{cases}$$

```
1 fatorial(a):  
2     se a == 1:  
3         retorne 1;  
4     retorne a × fatorial(a - 1);
```

Algoritmo e seu tempo de execução, $T(n)$

```
1 fatorial(a):  
2     se a == 1:  
3         retorne 1;  
4     retorne a × fatorial(a - 1);
```

Qual o tempo de execução, $T(n)$?

Algoritmo e seu tempo de execução, $T(n)$

```
1 fatorial(a):  
2     se a == 1:  
3         retorne 1;  
4     retorne a × fatorial(a - 1);
```

Qual o tempo de execução, $T(n)$?

$$T(n) = \begin{cases} T(n-1) + O(1) & \text{se } n > 1 \\ O(1) & \text{se } n = 1 \leftarrow \text{caso base} \end{cases}$$

Método da substituição repetida

$$T(n) = \begin{cases} T(n-1) + O(1) & \text{se } n > 1 \\ O(1) & \text{se } n = 1 \leftarrow \text{caso base} \end{cases}$$

Qual o limite assintótico?

- Expandir
- Expressar por somatória $\rightarrow$ padrão
- Resolver padrão

Método da substituição repetida

$$T(n) = \begin{cases} T(n-1) + O(1) & \text{se } n > 1 \\ O(1) & \text{se } n = 1 \leftarrow \text{caso base} \end{cases}$$

Qual o limite assintótico?

- Expandir
- Expressar por somatória $\rightarrow$ padrão
- Resolver padrão

$$\begin{aligned} T(n) &= T(n-1) + 1 \\ &= [T(n-2) + 1] + 1 \\ &= [T(n-3) + 1] + 2 \\ &= \dots \\ &= T(1) + (n-1) \\ &= 1 + (n-1) \\ &= n \\ &= O(n) \end{aligned}$$

Resolução do padrão: Séries

Aritimética: $\sum_{k=1}^n k = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$

Geométrica: $\sum_{k=0}^n x^k = 1 + x + x^2 + \dots + x^n = \frac{x^{n+1}}{x-1} (x \neq 1)$

Harmônica: $\sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \dots + \frac{1}{n} \approx \ln n$

Outras:

$$\sum_{k=1}^n \lg k \approx n \lg n$$

$$\sum_{k=1}^n c = cn$$

- Divisão e Conquista (e Intercalação)

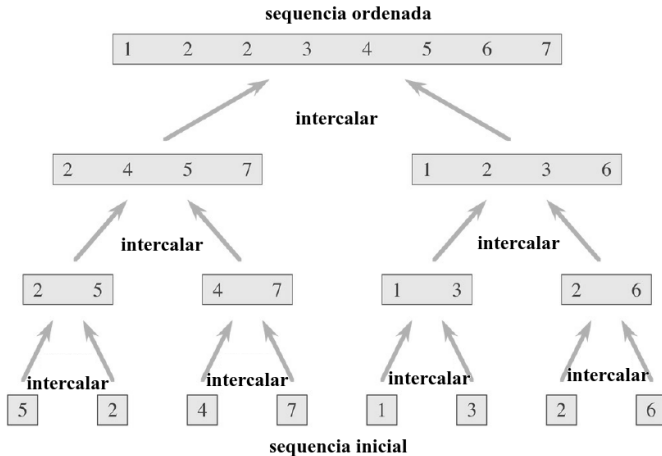


Figura: Exemplos da Ordenação por intercalação. Adaptado de: CORMEN, Thomas H. et. al. Introduction to Algorithms. 3 ed. The MIT Press, 2009

Mergesort

5	2	4	7	1	3	2	6
---	---	---	---	---	---	---	---

Mergesort

5	2	4	7	1	3	2	6
---	---	---	---	---	---	---	---

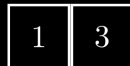
5	2	4	7
---	---	---	---

1	3	2	6
---	---	---	---

Mergesort



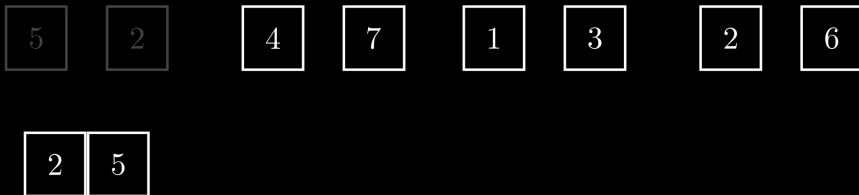
Mergesort



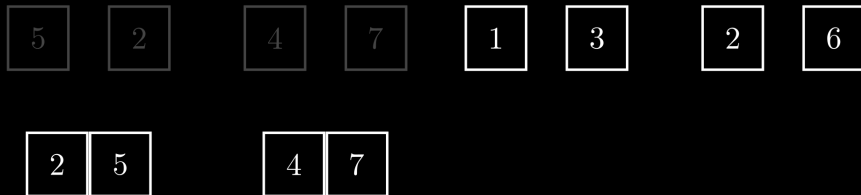
Mergesort



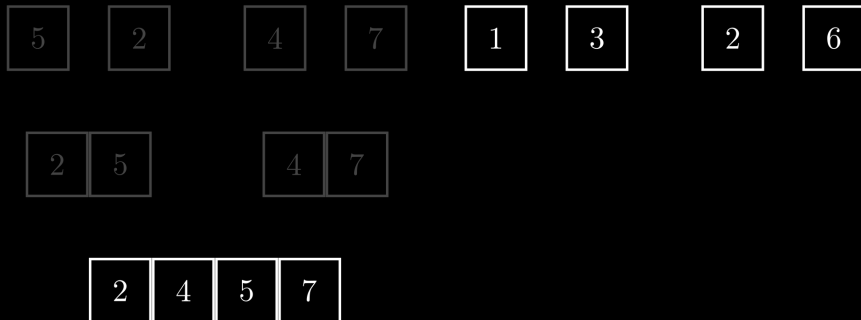
Mergesort



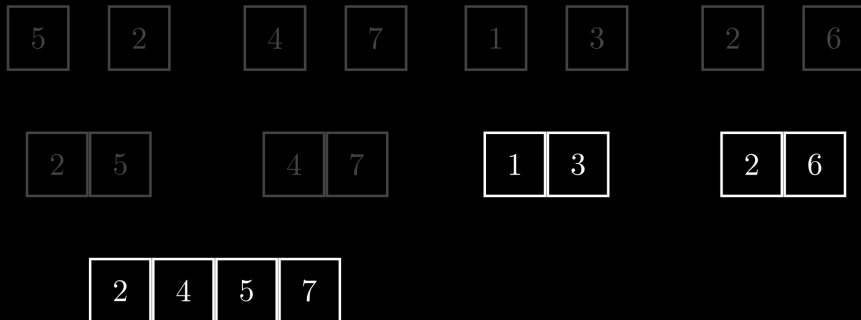
Mergesort



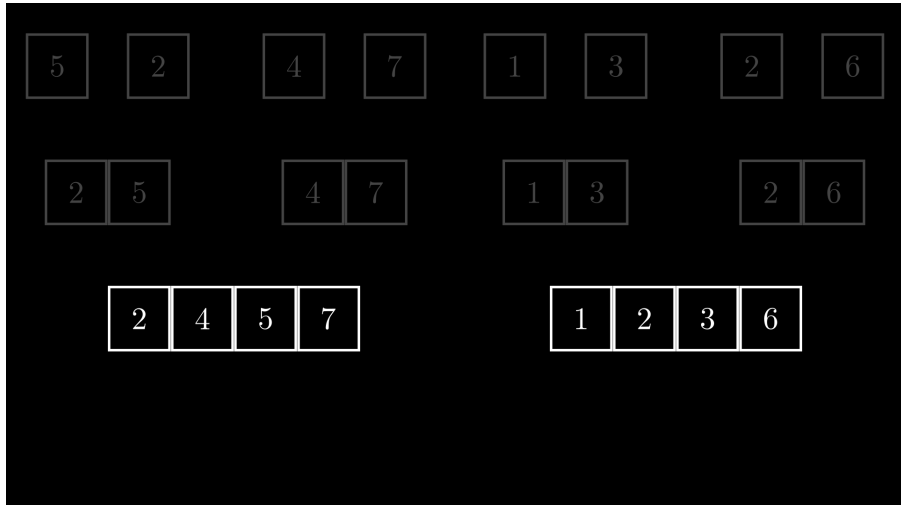
Mergesort



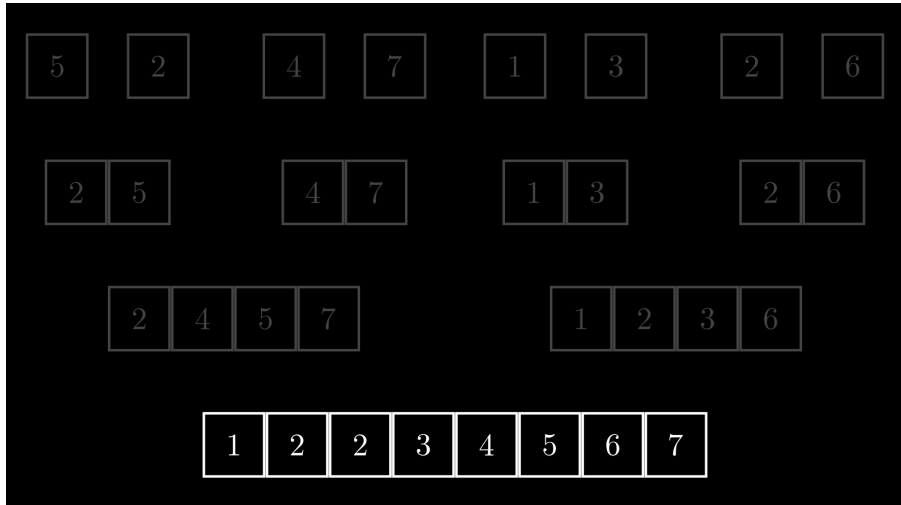
Mergesort



Mergesort

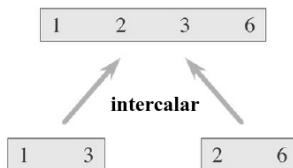


Mergesort



Algoritmo

```
1 merge(A, p, q, r):
2    $n_1 = q - p + 1$ 
3    $n_2 = r - q$ 
4    $E[1..n_1] = A[p..q]$ 
5    $D[1..n_2] = A[(q+1)..r]$ 
6    $E[n_1 + 1] = \infty$ 
7    $D[n_2 + 1] = \infty$ 
8    $i = 1$ 
9    $j = 1$ 
10  para  $k = p$  até  $r$ :
11      se  $E[i] \leq D[j]$ :
12           $A[k] = E[i]$ 
13           $i++$ 
14      senão:
15           $A[k] = D[j]$ 
16           $j++$ 
17
```



Adaptado de: CORMEN, Thomas H. et. al. Introduction to Algorithms. 3 ed. The MIT Press, 2009

Algoritmo e seu tempo de execução, $T(n)$

```
1 merge-sort(A, p, r):  
2     se p < r:  
3         q = ⌊ (p+r)/2 ⌋  
4         merge-sort(A, p, q)  
5         merge-sort(A, q+1, r)  
6         merge(A, p, q, r)
```

Chamar merge-sort(A, 1, len(A))

Algoritmo e seu tempo de execução, $T(n)$

```
1 merge-sort(A, p, r):  
2     se p < r:  
3         q = ⌊ (p+r)/2 ⌋  
4         merge-sort(A, p, q)  
5         merge-sort(A, q+1, r)  
6         merge(A, p, q, r)
```

Chamar merge-sort(A, 1, len(A))

Qual o tempo de execução, $T(n)$?

Algoritmo e seu tempo de execução, $T(n)$

```
1 merge-sort(A, p, r):  
2     se p < r:  
3         q = ⌊ (p+r)/2 ⌋  
4         merge-sort(A, p, q)  
5         merge-sort(A, q+1, r)  
6         merge(A, p, q, r)
```

Chamar merge-sort(A, 1, len(A))

Qual o tempo de execução, $T(n)$?

$$T(n) = \begin{cases} 2T(n/2) + O(n) & \text{se } n > 1 \\ O(1) & \text{se } n = 1 \end{cases}$$

Algoritmo e seu tempo de execução, $T(n)$

```
1 merge-sort(A, p, r):  
2     se p < r:  
3         q = ⌊ (p+r)/2 ⌋  
4         merge-sort(A, p, q)  
5         merge-sort(A, q+1, r)  
6         merge(A, p, q, r)
```

Chamar merge-sort(A, 1, len(A))

Qual o tempo de execução, $T(n)$?

$$T(n) = \begin{cases} 2T(n/2) + O(n) & \text{se } n > 1 \\ O(1) & \text{se } n = 1 \end{cases}$$

Qual o limite assintótico?

- Árvore de recursão
- Método da substituição
- Teorema mestre

Árvore de recursão

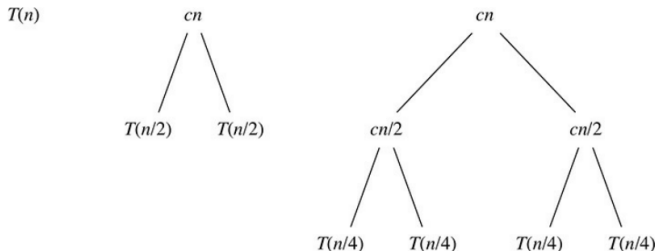


Figura: Árvore de recursão da ordenação por intercalação. Adaptado de: CORMEN, Thomas H. et. al. Introduction to Algorithms. 3 ed. The MIT Press, 2009

$$T(n) = \begin{cases} 2T(n/2) + O(n) & \text{se } n > 1 \\ O(1) & \text{se } n = 1 \end{cases}$$

Árvore de recursão

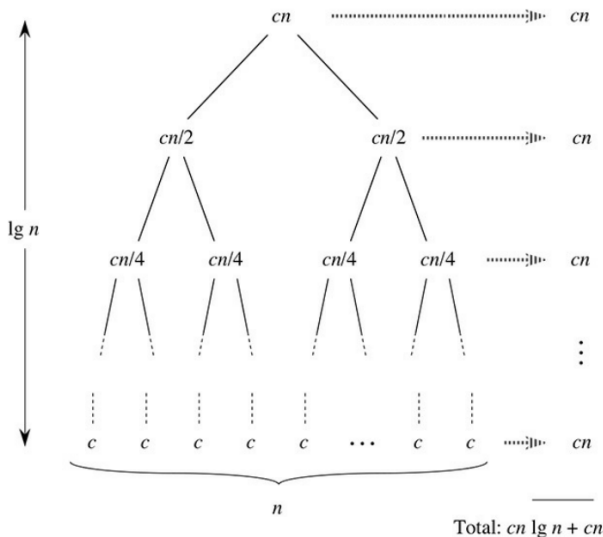


Figura: Árvore de recursão da ordenação por intercalação. Adaptado de: CORMEN, Thomas H. et. al. Introduction to Algorithms. 3 ed. The MIT Press, 2009

Método da substituição

- Arriscar um palpite para a forma da solução
- Usar indução para determinar as constantes e mostrar que a solução funciona
- Exemplo $T(n) = 2T(\lfloor n/2 \rfloor) + n$; tentar $T(n) = O(n \lg n)$
- Provar que $T(n) \leq cn \lg n$ para uma escolha apropriada da constante $c > 0$.
- Condição se mantém $\forall m < n$, especialmente para $m = n/2$

Método da substituição

- Arriscar um palpite para a forma da solução
- Usar indução para determinar as constantes e mostrar que a solução funciona
- Exemplo $T(n) = 2T(\lfloor n/2 \rfloor) + n$; tentar $T(n) = O(n \lg n)$
- Provar que $T(n) \leq cn \lg n$ para uma escolha apropriada da constante $c > 0$.
- Condição se mantém $\forall m < n$, especialmente para $m = n/2$

$$\begin{aligned}T(n) &\leq 2(c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \\&\leq cn \lg(n/2) + n \\&= cn \lg n - cn \lg 2 + n \\&= cn \lg n - cn + n \\&\leq cn \lg n\end{aligned}$$

$$\text{se } c \geq 1$$

Condições de contorno: $T(1) = 1 \rightarrow T(1) \leq c \cdot 1 \lg 1 = 0$

$$T(n) \leq cn \lg n \text{ para } n \geq n_0, \text{ logo } n_0 = 2$$

Teorema mestre

$$T(n) = aT(n/b) + f(n)$$

$a \geq 1$ e $b > 1$

- ❶ Se $f(n) = O(n^{\log_b a - \epsilon})$ para $\epsilon > 0$ então $T(n) = \Theta(n^{\log_b a})$
- ❷ Se $f(n) = \Theta(n^{\log_b a} \log^k n)$ então $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
- ❸ Se $f(n) = \Omega(n^{\log_b a + \epsilon})$ para:
 - ▶ $\epsilon > 0$
 - ▶ Se $af(n/b) \leq cf(n)$ para $c < 1$ e $\forall n$ suficientemente grande então $T(n) = \Theta(f(n))$

Exercício

- ❶ $T(n) = 9T(n/3) + n$
- ❷ $T(n) = T(2n/3) + 1$
- ❸ $T(n) = 3T(n/4) + n \lg n$
- ❹ $T(n) = 2T(n/2) + n \lg n$

Teorema mestre

Em resumo, se verifica se $f(n)$ é polinomialmente menor, maior ou igual à $n^{\log_b a}$.

Condição de regularidade no caso 3

é satisfeita pela maioria das funções polinomialmente limitadas que encontraremos.

Análise de Complexidade de Algoritmos Recursivos e Divisão e Conquista: mergesort

Yuri Kaszubowski Lopes

UDESC